

EXPERIMENT 3: Implement A* Search algorithm

EVALUATION

Writeup (3M)	
Execution (5M)	
Viva Voce (2M)	
Total Marks (10M)	
Faculty Signature and Date	

3.1. AIM

To implement and demonstrate the **A* Search Algorithm** for finding the optimal path from an initial state to a goal state using heuristic information in Python.

3.2. PROBLEM DESCRIPTION

The **A*** (A-Star) search algorithm is an informed search algorithm widely used in Artificial Intelligence for path finding and graph traversal. It combines the advantages of **Uniform Cost Search** and **Greedy Best First Search**.

The algorithm selects the next node based on the evaluation function:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ = cost from start node to current node
- $h(n)$ = estimated cost from current node to goal (heuristic)
- $f(n)$ = total estimated cost of solution through node n

The node having the lowest $f(n)$ value is expanded first.

3.3. CONSTRAINTS AND ASSUMPTIONS

Constraints

1. The graph must have non-negative edge costs.
2. The heuristic function should be admissible (should not overestimate the actual cost).
3. Nodes already explored should not be revisited unnecessarily.
4. The goal state must be reachable.

Assumptions

1. The graph is finite.
2. Path cost between nodes is known.
3. Heuristic values are predefined.
4. The optimal path exists.

3.4. ALGORITHM (A* Search Algorithm)

1. Create an OPEN list (priority queue) and CLOSED list.
2. Insert the start node into OPEN list.
3. Assign:
 - $g(\text{start}) = 0$
 - $f(\text{start}) = h(\text{start})$
4. Repeat until OPEN list is empty:
 - Select node with lowest $f(n)$.
 - If it is the goal node, stop.
 - Move node to CLOSED list.
 - Generate all successor nodes.
 - Compute:

$$g(\text{successor}) = g(\text{current}) + \text{cost}$$

$$f(\text{successor}) = g(\text{successor}) + h(\text{successor})$$

- If successor not in OPEN or CLOSED, add it to OPEN.
 - If already present with higher cost, update it.
5. Trace the path from goal to start.

3.5. PROGRAM

<https://github.com/mayurjambit/BAD402>

```
import heapq

# Graph representation
graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 3), ('E', 1)],
    'C': [('F', 5)],
    'D': [],
    'E': [('F', 2)],
    'F': []
}

# Heuristic values
heuristic = {
    'A': 6,
    'B': 4,
    'C': 4,
```

```
'D': 3,  
'E': 2,  
'F': 0  
}  
"  
  
graph = {  
    'A': [('B', 2), ('C', 4)],  
    'B': [('D', 7), ('E', 3)],  
    'C': [('F', 5), ('G', 6)],  
    'D': [('H', 4)],  
    'E': [('H', 2), ('I', 6)],  
    'F': [('I', 3)],  
    'G': [('I', 4)],  
    'H': [('J', 5)],  
    'I': [('J', 2)],  
    'J': []  
}  
  
heuristic = {  
    'A': 10,  
    'B': 8,  
    'C': 9,  
    'D': 7,  
    'E': 5,  
    'F': 6,  
    'G': 6,  
    'H': 4,  
    'I': 2,  
    'J': 0  
}  
"
```

```
def astar(start, goal):  
    open_list = []  
    heapq.heappush(open_list, (0, start))  
  
    g_cost = {start: 0}  
    parent = {start: None}  
  
    closed = set()  
  
    while open_list:  
        _, current = heapq.heappop(open_list)  
  
        if current == goal:  
            path = []  
            while current:  
                path.append(current)  
                current = parent[current]  
            return path[::-1]  
  
        closed.add(current)  
  
        for neighbor, cost in graph[current]:  
            if neighbor in closed:  
                continue  
  
            new_g = g_cost[current] + cost  
  
            if neighbor not in g_cost or new_g < g_cost[neighbor]:  
                g_cost[neighbor] = new_g
```

```
f_cost = new_g + heuristic[neighbor]
heapq.heappush(open_list, (f_cost, neighbor))
parent[neighbor] = current
```

```
return None

path = astar('A', 'F')
print("Optimal Path:", path)
```

3.6. OUTPUT